Etapa 2: Elección del Patron de Lógica de Negocio

Se analizarán tres patrones de arquitecturas (Transaction Script, Table Module y Domian Model) basándonos en la complejidad de las reglas de negocio de FerreMax documentadas en la unidad 2, los cambios esperados y el manejo de los estados de la empresa.

A continuación, se presenta la tabla comparativa:

Criterio del caso	Transcripción Script	Table module	Domain Model	Evidencia en U2
Complejidad de reglas	Agruparía todas las reglas de descuentos (5% o 10%) y vigencias (3 o 5 días) en un solo método procedimental gigante, volviéndolo difícil de mantener.	Separaría la lógica por tablas de base de datos (ej. Cotizacion, Cliente), pero la validación de negocio seguiría ligada a los datos crudos y no al comportamiento.	Permite crear objetos ricos (ej. Cotizacion, ClienteEmpresarial) que encapsulan sus propias reglas para calcular descuentos y fechas de vencimiento de forma natural.	BR005, BR006, US-005, US-006.
Cambios esperados	Si cambian las políticas de entrega a domicilio (zonas de riesgo, horarios), habría que rastrear y modificar múltiples scripts dispersos que procesen ventas.	Facilitaría agregar columnas a la tabla de envíos, pero complicaría agregar nuevas reglas de negocio condicionales que involucren múltiples tablas a la vez.	Las políticas de entrega se pueden modelar como estrategias o servicios de dominio aislados, permitiendo escalar las reglas sin afectar el resto del sistema.	BR011, US-011.
Interacción entre conceptos	Una venta a crédito requeriría un script que llame a inventario, luego a cobranza y luego a rutas, generando un alto acoplamiento.	La lógica de una venta parcial obligaría a que el módulo de "Ventas" tenga que conocer e interactuar íntimamente con la estructura de las tablas de "Inventario" y "Backorder".	Un Aggregate de Orden puede orquestar la interacción fluida con Inventario y LineaDeCredito, validando que todo se cumpla antes de persistir los cambios.	BR007, BR012, US-007, US-012.
Estados e invariantes	El control del estado de la mercancía ("en tránsito", "reservado") dependería de que el programador no olvide actualizar un campo en la base de datos en cada script.	Manejaría los cambios de estado actualizando registros de una tabla, pero no protegería las transacciones si otro módulo altera la base directamente.	Protege las invariantes de forma estricta: un objeto Cliente no permitirá procesar una venta si su estado interno es "Crédito suspendido", garantizando consistencia.	BR008, BR013, US-008, US-013.

Criterio adicional: Concurrencia y reservas en tiempo real	Bloquearía tablas completas o registros sueltos de manera ineficiente durante la reserva de stock, afectando a las 3 sucursales.	Dependería fuertemente de los bloqueos de base de datos a nivel de tabla para evitar la doble venta, limitando la escalabilidad del sistema.	El modelo de dominio gestiona la concurrencia a nivel de Aggregate Root (ej. Producto), asegurando que la reserva en tiempo real sea atómica y segura.	BR001, US-001, US-019

La argumentación de la decisión que se tomó:

Para el caso de FerreMax, se descarta *Transaction Script* porque el sistema no es un simple CRUD. Las operaciones principales desencadenan múltiples validaciones y efectos secundarios. Por ejemplo, concretar una venta requiere descontar stock en tiempo real (BR001), validar líneas de crédito y saldos vencidos (BR007), y potencialmente generar un backorder si el pedido es parcial (BR012). Implementar esto como scripts transaccionales resultaría en código duplicado y propenso a errores, especialmente cuando el negocio escale.

Table Module también se descarta porque, aunque organiza mejor el código alrededor de las tablas de la base de datos, FerreMax requiere proteger invariantes estrictas que cruzan múltiples conceptos. Por ejemplo, bloquear ventas cuando el límite de crédito se excede (BR008) o asegurar que las cotizaciones vencidas no se apliquen sin la excepción de un gerente (BR004, BR009). El modelo tabular no maneja bien esta lógica comportamental compleja.

La elección de **Domain Model** es la más adecuada para FerreMax por las siguientes razones respaldadas en el análisis de requerimientos:

- Proteccion de Invariantes Complejas**:** FerreMax dicta que una mercancía transferida debe marcarse como "en tránsito" y bloquearse para la venta hasta que la otra sucursal confirme su recepción (BR013). Un modelo de dominio permite modelar la Transferencia y el Inventario como entidades con estados claros que impiden alteraciones indebidas.
- Encapsulamiento de Logica Condicional: La asignación de la vigencia de cotizaciones (5 días regular, 3 días empresarial) (BR005) y los descuentos automáticos (5% y 10%) (BR006) dejarán de ser sentencias if/else dispersas en controladores. Estas reglas vivirán dentro de la entidad Cotizacion, calculándose intrínsecamente según el tipo de cliente.
- Transiciones Seguras y Autorizaciones: Requerimientos como la excepción para ventas sin stock (BR016) o la autorización de excepciones de crédito (BR009) requieren un manejo de estados rico. El modelo de dominio permite

que entidades como Venta solo avancen de estado si se inyecta un objeto de AutorizacionGerencial.

Implementar el Modelo de Dominio exige una curva de aprendizaje mayor para el equipo, ya que requiere separar estrictamente el código de infraestructura (bases de datos, UI) del núcleo del negocio. Nos obligará a mapear objetos de memoria a tablas relacionales utilizando un ORM o repositorios personalizados en las siguientes etapas, lo cual añade una capa extra de complejidad técnica en comparación con las consultas SQL directas. Sin embargo, este costo se justifica plenamente para erradicar el problema crítico actual del negocio: las ventas de mercancía ya comprometida (BR019).